1.ABSTRACT

This study was motivated by growing trend in students and retail traders who tries to build algorithmic trading by technical analysis and machine learning models, they often expect to generate higher returns by relatively low efforts. Many people think that their python-based models can beat the market, but reality is different.

In this research by using daily OHLCV data we tested two actively used trading strategies on Nifty 50 index from Jan 2021 to Jan 2024. The first strategy is traditional 20/50 days moving average (SMA), while the other one was random forest trained on 17 technical indicators.

In results strategies clearly underperformed. After adding transaction cost (0.2% round trip) the MA gave only 12.37% results with 17 trades compared to 55.02% from simple buy and hold strategy. The random forest model suffered from severe outfitting, recording 100% training accuracy but when applied in real world accuracy dropped to only 50%, overall delivering net result of 5.34% with 50 trades

Both the strategies failed because they relied heavily on publicly available data. It caused lagging signals in moving average, and severe overfitting in random forest, also the high transactional cost by frequent trading.

The key takeaway is that students and retail traders who tries to beat market by using simple technical indicators or basic ML models based on publicly available data are unlikely to succeed. Passive long-term investing through low-cost index funds or ETFs appears to be a more reliable approach.

Keywords: Nifty 50, Moving Average Crossover, Random Forest, Technical Analysis, Machine Learning in Finance, Market Efficiency, Transaction Costs

2. INTRODUCTION

2.1 The Problem: Students and Trading Bots

Encouraged by a large number of YouTube videos, online courses, and easy availability of Python libraries and machine learning tools, many students and retail traders have started to build their own algorithmic trading strategies. Many people believe that by developing simple trading bots, they can generate huge profits from the stock market. They think they can predict stock price movements and easily beat the market.

There is a widespread belief that "ML will predict the market." However, reality is far more different and disappointing. These trading strategies disappoint when they don't generate expected profits or make losses when applied in real life. Most importantly, traders fail to understand why their carefully built systems don't work as expected.

2.2 Why This Research Matters

The gap between expectations and reality makes it really important to rigorously test whether these popular trading strategies actually perform well in practice. With thousands of students spending their time and effort developing these trading strategies, there is a need for studying the true effectiveness of these approaches.

Most existing studies tend to highlight the potential of ML in finance, often showing impressive in-sample results. However, only a few papers study the limitations and failures of these strategies when tested on unseen data with realistic costs. This research attempts to fill this gap by presenting an honest evaluation of two commonly used strategies and explaining why they underperform.

2.3 Research Question

The central question this paper seeks to answer is:

Can simple technical analysis and basic machine learning strategies, built only on publicly available data, consistently outperform a passive buy-and-hold approach on the Nifty 50 index?

2.4 What This Paper Does

To address this question, the study evaluates two popular active trading approaches on the Nifty 50 index using daily data from January 2021 to January 2024. The first is a traditional 20-day versus 50-day simple moving average (SMA) crossover strategy. The second is a Random Forest classifier trained on 17 technical indicators to predict daily price direction.

What makes this study different is its emphasis on realism: it includes realistic transaction costs (0.2% round-trip cost per transaction), uses proper chronological train-test split, and reports both failures and successes transparently. Rather than presenting promising results, this paper seeks to understand why these strategies fail in practice.

2.5 Structure of the Paper

The remainder of the paper is organized as follows: Section 2 describes the methodology, Section 3 presents the empirical results, Section 4 provides a detailed analysis of the findings, and Section 5 concludes with key lessons and practical implications.

3.METHODOLOGY

3.1 DATA COLLECTION

I used Nifty 50 index data for this research. Nifty 50 is index of top 50 companies listed on National Stock Exchange Index (Ticker symbol: ^NSEI). This data is obtained from Yahoo finance using yfinance library of python.

- Time period of this data set: 1 January 2021 to 1 January 2024.

- This data set include 741 trading days.
- This dataset provides OHLCL (Open,High,Low,Close,Volume) of each trading day.

3.2 Strategy 1: Moving average crossover

The first strategy I used is moving average crossovers. It is very traditional technical analysis approach this strategy uses crossovers to identify buy or sell signals

Implementation:

I calculate two rolling moving averages.

- Short-term moving average: 20 days SMA.
- Long-term moving average: 50 days SMA.

Trading rules:

- Golden cross: when 20 days SMA crosses above the 50 days SMA it is called golden crossover. It indicates buy signal.
- Death cross: when 20 days SMA crosses below the 50 days SMA it is called death cross. It indicates sell signal.
- Positions: whenever the signal is (1) we invest fully in stocks, when signal is 0, we are fully into cash.

Calculations:

- To calculate moving averages for n days we use formula

  (price[t] + price[t-1] +.....+ price[t-n+1])/n

  Here t represents today so price[t] means price today and price [t-1] means price yesterday and so on.

Signals:

If MA (20) > MA (50) signals 1 (BUY), else it is 0.

3.3 Strategy 2: Random Forest Machine Learning

In second strategy I deployed a random forest classifier to predict daily trading signals.

Features engineering:

I used 17 technical indicators as technical inputs

- Price based features
  - 1-day return, 5-days return, 20-days return.
  - 5-day moving average, 10-day moving average, 20-day moving average, 50-day moving average.

    - Moving averages ratio (5/20), (10/50).
- Momentum indicators
    - 5-day and 20-day momentum.
    - RSI (relative strength indicator) 14-day period.
- Volatility measures
    - 5-day and 20-day rolling volatility measures
- Volume parameters
    - Raw volume
    - 20 day's volume moving average
    - Volume ratio (current/average)

Target variable:

If tomorrow's closing price is greater than today's price(price[t+1] > price[t] than it will signal (1) otherwise it will be (0). So, the target variable is just binary classification, it will predict 1 if price goes up otherwise it will be zero.

Model specification:

So, for this trading strategy I used algorithm which is Random Forest classifier (scikit – learn). This forest have 100 decision trees. Depth of this is forest is 10 trees. Other parameters include default settings of (scikit-learn).

Train / test split :
This data includes total of 692 samples. In which 70% of data (484 samples) were used for training the model and rest 30% of data (208 samples) were used for testing the data. The data which is used for training the model was from JAN 2021-FEB 2023 and was tested on datasets of dates MAR 2023-DEC 2024.

Trading rules:

So, whenever model predict that the price will go up it gives (prediction = 1). And when model predicts the price will go down (prediction = 0)

3.4 Back-testing framework:

All the strategies were back tested on historical data with following setup.

Initial capital:

I used capital of rupees 1,00,000 (paper money) not real money for this research.

Transaction cost:

Indian equity market includes various transactional fees

- Brokerage fees: ~0.03%
- Securities transaction tax: 0.025%
- Exchange fees: 0.00325%
- GST on brokerage :18% of brokerage

Total estimated cost on per trade equals 0.1% and full round-trip trade (BUY + SELL) = 0.2%.

Back-testing methodology:

For each trading day, I followed these steps

- Check the strategy signal (BUY OR SELL).
- If strategy changed, execute the trade and deduct transaction cost.
- calculate current portfolio value (cash + holdings).
- Track total number of trades.

At the end I compared final portfolio value to initial portfolio value to calculate returns.

Benchmark comparison:

We also calculate returns of simple buy & hold strategy for comparison. Buy stock at first closing day and hold until last day. No intermediate trades (zero transactional fees).

4. RESULTS

This section presents the empirical findings from back testing both strategies on Nifty 50 data from January 2021 to January 2024.

4.1 Moving Average Crossover Results

I generated 17 trades based on this strategy. Table 1 compares performance across three scenarios: buy-and-hold, MA strategy without transaction costs, and MA strategy with realistic transaction costs.

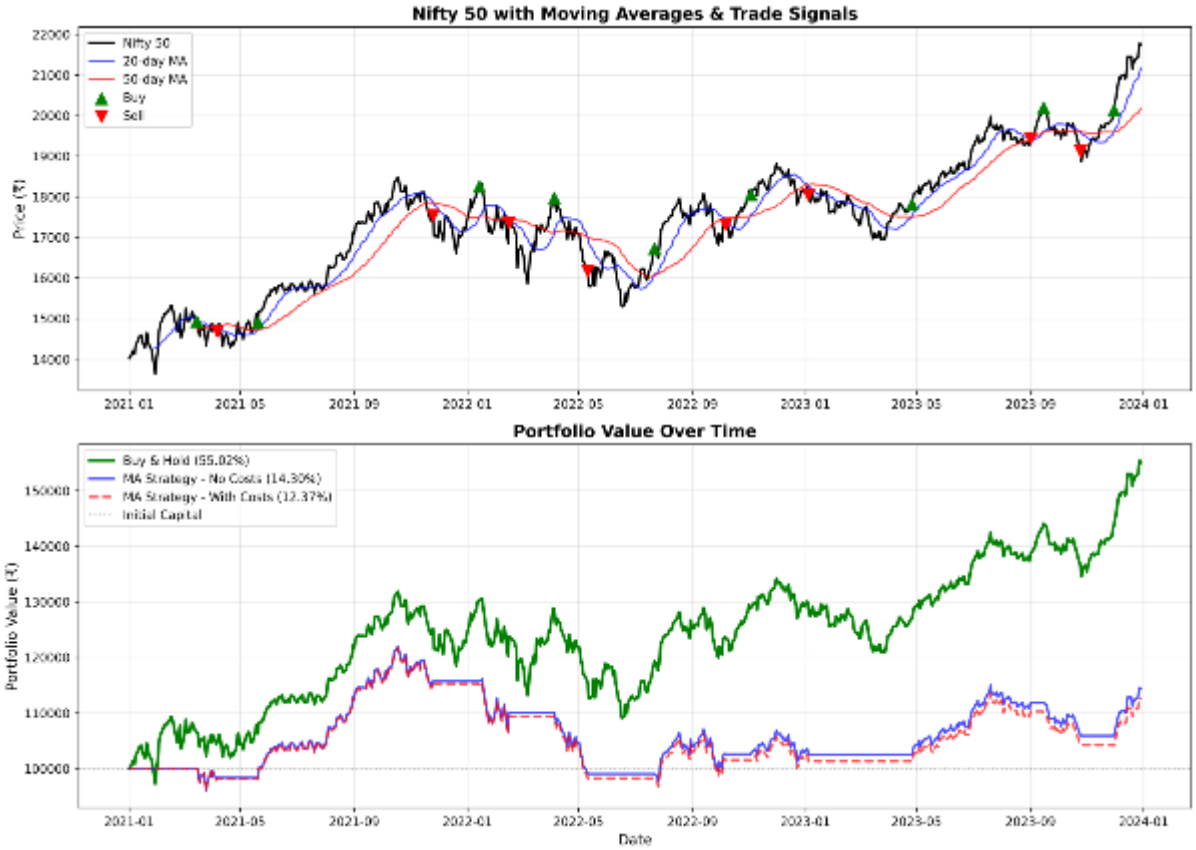
Table 1: Moving Average Strategy Performance

Metric	Buy & Hold	MA Strategy (No Costs)	MA Strategy (With Costs)
Total Return	55.02%	14.30%	12.37%
Sharpe Ratio	0.68	-0.10	-0.16
Maximum Drawdown	-17.23%	-19.93%	-20.41%

Number of Trades	0	17	17
Transaction Cost Impact	-	-	-1.93%

What I can clearly see is that this strategy underperformed simple buy-and-hold by 42.65%. The MA strategy achieved only 14.30% return compared to 55.02% for buy-and-hold. When I include realistic transaction costs (0.1% per trade), returns dropped from 14.30% to 12.37%.

The Sharpe ratio is also negative (-0.16 with costs), which indicates the strategy delivered negative risk-adjusted returns. Maximum drawdown was also worse than buy-and-hold (-20.41% vs -17.23%), which means the strategy experienced larger losses during market downturns.



4.2 Random Forest Results

4.2.1 Model Performance

Table 2 shows the Random Forest classifier's accuracy on training and testing datasets.

Dataset	Accuracy	Samples
Training	100.00%	484
Testing	50.48%	208
Overfit Gap	49.52%	-

While training the dataset, I got perfect 100% accuracy, correctly predicting every price direction in the training set. However, when applied to real-world scenarios where the model sees unseen data, accuracy dropped to 50.48% - which is barely better than random guessing (50%).

This 49.52 percentage point gap demonstrates severe overfitting. The model memorized specific patterns from training data rather than learning generalizable relationships.

4.2.2 Feature Importance

The Random Forest model identified these top 5 most important features:

Table 3: Top 5 Feature Importances

Rank	Feature	Importance Score
1	Returns-1day	0.0899
2	volume	0.0638
3	Volatility-20 day	0.0624
4	RSI	0.0616
5	Volatility-5day	0.0610

Despite identifying these features as "important," the model's 50% test accuracy proves that these features have no real predictive power for future price movements.

5.2.3 Trading Performance

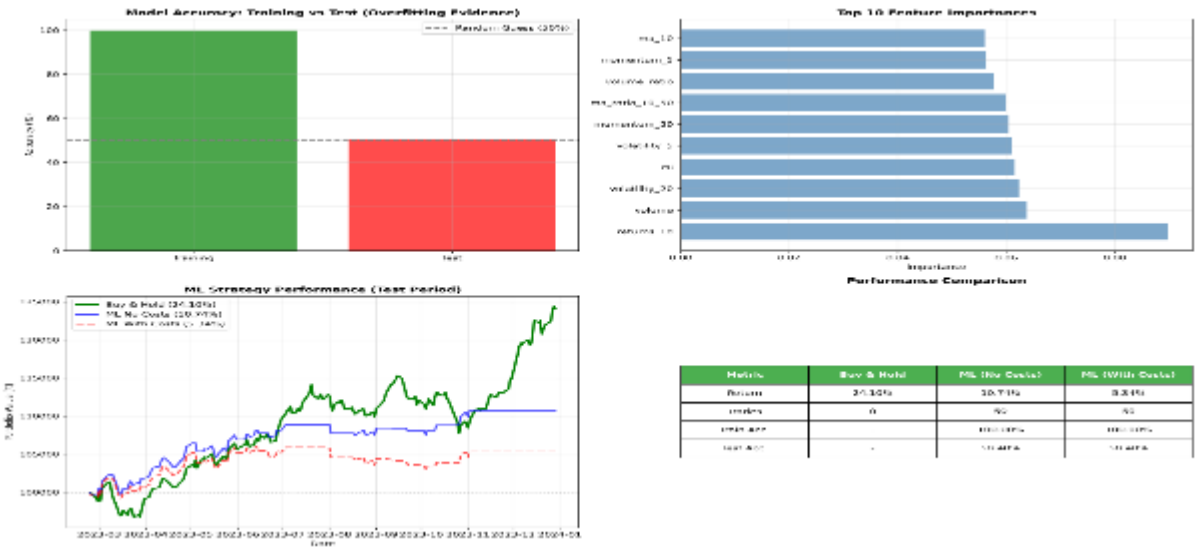
This strategy produced 50 trades. Table 4 compares performance.

Table 4: Random Forest Strategy Performance (Test Period)

Metric	Buy & Hold	RF Strategy (No Costs)	RF Strategy (With Costs)
Total Return	24.10%	10.74%	5.34%
Number of Trades	0	50	50
Transaction Cost Impact	-	-	-5.40%

What I can see is that this strategy underperformed buy-and-hold by 18.76% on the test period. With 50 trades, transaction costs consumed 5.40% of returns - reducing performance from 10.74% to just 5.34%.

The high number of trades (50 in ~200 trading days = trading every 4 days) resulted from the model's random predictions generating frequent buy/sell signals.



4.3 Comparative Summary

Table 5: Strategy Comparison

Strategy	Return	Trades	Key Issue
Buy & Hold (Full Period)	55.02%	0	-
Moving Average	12.37%	17	Lagging signals
Random Forest	5.34%	50	Overfitting (100%→50%)
Buy & Hold (Test Period)	24.10%	0	-

Both strategies performed terribly. There is a massive gap between simple buy-and-hold strategies and our trading strategies. What surprises me more is that the Random Forest model only generated 5.34% returns despite being labelled as an advanced ML model.

5.ANALYSIS

This section analyses why both strategies failed. I will explain the specific reasons behind the poor performance of each strategy

5.1 Analysis of moving average strategy

5.1.1 Lagging indicator

The first thing is moving average are lagging indicator, in simple term, this strategy gives signals very late. For example, the price went from 14,500 to 15,500 but our trading strategy only gave us signal when price had already went to 15,200 which means I lost rupees 700 gains upfront and only scratched 300 rupees profit(15,200 to 15,500) Now when price went high investors start to think that this asset had went high so it won't up further that's why they start to sell their assets and price went down. In this whole cycle apparently, we didn't make any real profit.

5.1.2 No edge over other traders

The second main problem with this strategy is that everyone has access to index data, charts, and crossover signals. This is a traditional technical analysis approach that traders widely use these days.

Think about it: if everyone has access to the same data, everyone gets the same signals. Most traders lose money using these strategies, so you will likely lose money too. The reason is simple - you have no edge or advantage over other traders.

You can only make consistent profits when you have an edge over other trader. The people who have this edge are those with inside information - they know what actions a company will take before it becomes public. They make their trades, book their profits, and leave before these trading strategies even generate signals. By the time our strategy signals a buy, they have already made their profit.

This point seems too obvious, but most traders miss it because they are not doing trading - they are just gambling.

5.1.3 Transaction Costs and Returns Analysis

We had already discussed about transactional cost. Each transaction cost (0.1%) and we made roughly 17 transactions in these strategies so, we had to pay significant transactional cost also. The results were already very terrible, but this transactional cost acted like cherry on the cake for us.

Our 'Smart' trading algorithm generated 14.30% returns, but after deducting transaction costs, returns dropped to 12.37%. You might think 12.37% is not that bad but compare it to this a simple buy low sell high strategy what we normally call buy & hold generate 55.02%, now u can see the numbers are miserable. The gap between two is huge.

If u wanted 12.37% returns why you are doing trading just invest in mutual funds. A mutual fund on an average generates 14% returns if u had hired funds manager u will be making more profit.

5.2 Analysis of random forest

strategy 5.2.1 Overfitting problem

In this trading strategy, I used total of 692 samples, in which 484 samples were used for training the model and 208 samples were for testing the strategy. Now interesting thing that I found was while training the model I achieved accuracy of perfect 100% and that's why most students and retail traders think they can make lots of money from it or make project put it on their resume. But you must understand that this 100% accuracy is on training and not on testing. And when I started to test our model in real trading scenarios our smart ml model dropped to 50% only yes 50%.

Now u may why it happened even though we used advanced ml model for it and accuracy is only 50% which is same as random guessing. And you are right to question that. And then I started to think what could be reason for that as I was also amazed to get these results.

5.2.2 No real pattern exists

Now before diving into reasons for the 50% real accuracy u must understand that what does this 50% accuracy means. 50% accuracy means the model is random guessing.

And the reason for random guessing was that for model no real pattern exists. because what had we done we feed the data to model, so model instead of finding the general relationship it just memorized the data which has simply no use. I will explain this with example let's assume that we trained model so when (RSI = 65.3 and momentum = 120) give the signal buy (1) and what will model do it will memorize data when RSI = 65.3 and momentum = 120 I have to give buy signal.

But in real world scenarios that specific data would never repeat so when in real life data will be like (RSI = 63 and momentum = 115) now both the data are very similar for us but it's not similar for model for model it is new set of data that it had never seen in its training phase. So, now this model will start random guessing what signal to give so as I told u before 50% accuracy is what? It is random guessing and what's happening random guessing that's why accuracy is 50%.

So, for model no real pattern exists. also, trades are being made on public data as I explained that relying on public data for trading is not trading its gambling as u do not have any edge.

5.2.3 Too many transaction

In testing model made 50 trades and model was tested on ~200 days data so it means model made trade every fourth day, so transaction cost was high in this strategy we had to pay significant transactional cost it took almost half our returns were only 5.34% which are miserable if we compare this strategy to MA this strategy was complete failure. MA still produced some results but this strategy it was disaster.

5.3 The Common Problem

Both strategies (MA and RF) failed because they relied on the same public data. Everyone has access to this public data. When everyone can see the same signals, there is no advantage left.

The MA strategy failed because of lagging indicators and because everyone sees the same trading signals. The RF strategy failed because it memorized past data that never repeats exactly the same way again.

The lesson is clear: you can't consistently beat the market with data everyone has. Real profits come from having an edge - insider information, institutional resources, or superior technology - not from analysing public price data and technical indicators.

importance of proper out-of-sample testing and realistic cost inclusion before getting excited about any model. Overall, it shows how hard it is for individual investors to outperformer to market consistently.

6. CONCLUSION

6.1 Summary of Findings

This study tested two widely used trading strategies - Moving Average and Random Forest - against Nifty 50 data. The MA strategy used 20/50 MA crossovers, and the Random Forest used 17 technical indicators as inputs. Each transaction cost a realistic 0.2% for a full round trip. I used data from the time period January 2021 to January 2024.

The MA strategy made 17 trades and generated 12.37% returns. The RF strategy made 50 trades, producing 5.34% returns. In contrast, a simple buy-and-hold strategy over the full time period generated 55.02% returns. During the testing period alone, buy-and-hold produced 24.10% returns. Both active trading strategies massively underperformed and suffered from high transaction costs.

6.2 Key Lessons

The major lesson from this research is that models trained on public data cannot provide you an edge over other trader. The MA strategy reacts too slowly to price changes. Even ML models like Random Forest look perfect in training (100% accuracy) but fail completely when tested on new data (50% accuracy - random guessing).

Back testing results can be extremely misleading. What looks profitable in back tests often fails in real trading. Frequent trading kills returns through transaction costs - the RF strategy's 50 trades consumed nearly half of the gross returns.

6.3 Implications

For retail traders, these findings suggest that attempting to beat the market with common technical strategies or self-coded ML models carries high risk and often leads to disappointing results. For students, the study highlights the need for proper out-of-sample testing and realistic inclusion of costs before relying on any model. Overall, it shows how difficult it is for individual investors to consistently outperform the broader market using publicly available data.

6.4 What to Do Instead

Based on these findings, this study suggests that a simple passive investment strategy is more practical for most people. Rather than spending time and effort building trading strategies trained on public data, investors are likely to achieve better long-term results by investing regularly in low-cost index funds and ETFs that track Nifty 50.

For students interested in building a career in finance, the focus should shift toward developing core financial skills such as financial modelling, forecasting, Excel

proficiency, SQL, data visualization tools (like Power BI), and understanding corporate finance fundamentals. These skills are far more valuable and applicable in real-world roles like FP&A, investment banking, and equity research than building algorithmic trading strategies.

In conclusion, this study demonstrates that beating efficient markets like Nifty 50 using simple technical strategies or machine learning on public data is extremely difficult - if not impossible - for retail investors and students.

7.REFERENCES

Yahoo Finance. (n.d.). Nifty 50 Index (^NSEI). Retrieved from https://finance.yahoo.com

Python Software Foundation. (n.d.). yfinance library. Retrieved from https://pypi.org/project/yfinance/

Scikit-learn developers. (n.d.). Random Forest Classifier. Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html